Métricas e Indicadores de Calidad

TechnoStore · Catálogo E-commerce (PHP + MySQL) — Herramienta de calidad del Plan de Aseguramiento de Calidad (PAC)

Dato	Detalle
Proyecto	TechnoStore · tienda en línea con panel de administración y dinero digital simulado
Versión evaluada	v1.2 · código del archivo Nueva carpeta.zip (10/11/2026)
Fecha del documento	10/11/2026 · versión 1.0 del documento · **borrador para revisión del equipo**
Herramienta aplicada	Métricas e indicadores de calidad (enfoque Objetivo–Pregunta–Métrica y características ISO/IEC 25010)
Tipo de medición	Estática: conteo y lectura del código, de la base de datos y de los documentos del PAC (no se ejecutó la tienda)
Equipo	Danna Michelle Farfan Ramírez, Cristian Camilo Alegria Rivera, Kevin Camilo Caballero Oliveros, Deiner Smit Limas Lavao y Juan Manuel Silva Castañeda
Programa	Análisis y Desarrollo de Software (ADSO) · Ficha 3407799 · SENA

Resumen ejecutivo

Se definieron **29 indicadores** (18 de producto y 11 de proceso). Se pudieron medir 28; 1 quedó **sin medir** por falta de evidencia (rendimiento). Resultado: **5 cumplen**, **12 están en riesgo** y **11 no cumplen**. El **Índice de Calidad Global (ICG) es 39,3 %**, frente a la meta de 80 % para aprobar la entrega.

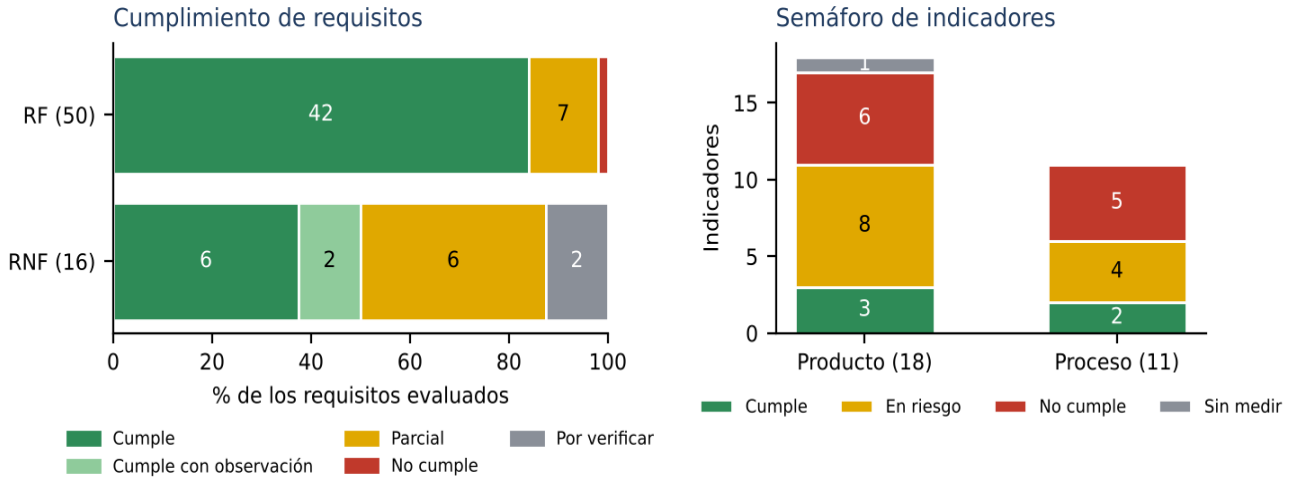


Figura 1. Cumplimiento de requisitos (Verificación de RF y evaluación de RNF) y semáforo de los indicadores.

Lo que está bien: el control de acceso está centralizado y cubre las 35 acciones no públicas; todas las consultas usan sentencias preparadas; la lógica de stock y saldo se ejecuta en transacciones con bloqueo (6 usos de FOR UPDATE) y los movimientos de dinero dejan trazabilidad; las 17 pruebas manuales pasan.

Lo que preocupa: sigue en el código la clave provisional 123456 (INC-05); no hay protección CSRF, flags de cookie ni límite de intentos de login; hay un riesgo de XSS en los botones de eliminar del panel; existen dos versiones distintas de bdcatalogo.sql; el puerto por defecto de la base de datos (8080) rompe una instalación limpia; y las pruebas manuales cubren solo 8 de 20 RF y **no detectaron ninguno** de los 7 errores registrados.

1. Objetivo, alcance y método

Objetivo. Medir de forma objetiva la calidad del producto TechnoStore y de su proceso de aseguramiento (pruebas, revisión de código y gestión de defectos), para que la decisión de entrega se apoye en datos y no solo en la percepción del equipo.

Alcance. Código (index.php, api.php, reporte.php, xlsx.php, mailer.php, config.php, módulo feedback, app.js, hojas de estilo), base de datos (bdcatalogo.sql) y documentos del PAC: SRS, Control de Calidad Técnico y Gestión de Versiones, Verificación de RF y evaluación de RNF.

Método. A cada objetivo de calidad se le asocian preguntas y, para cada pregunta, uno o más indicadores con fórmula, meta y fuente. Los indicadores de producto se agrupan según las características de ISO/IEC 25010.

Objetivo	Pregunta que responde	Indicadores
O1 · Funcionalidad completa y correcta	¿Qué porcentaje de los requisitos funcionales cumple lo especificado?	IP-01, IP-02, IP-17
O2 · Datos y accesos protegidos	¿Están protegidas las acciones, las contraseñas y las pantallas de administración?	IP-03 a IP-07
O3 · Confiabilidad e integridad	¿Se cumplen los requisitos no funcionales y es coherente el esquema de datos?	IP-08, IP-09
O4 · Mantenibilidad	¿Se puede entender, probar y modificar el código con poco riesgo?	IP-10 a IP-14
O5 · Usabilidad, accesibilidad y rendimiento	¿Funciona bien en pantallas pequeñas, es accesible y responde rápido?	IP-15, IP-16, IP-18
O6 · Proceso de calidad efectivo	¿Las pruebas, revisiones y el registro de errores encuentran y cierran los problemas antes de entregar?	IR-01 a IR-11

Escala del semáforo. Cada indicador se compara con su meta. Las metas provienen del SRS cuando existen (p. ej. 500 ms p95, 360 px, WCAG 2.1 AA); las demás son **metas propuestas** para esta versión y pueden ajustarse en la revisión del equipo.

Estado	Criterio
Cumple	Alcanza la meta.
En riesgo	No alcanza la meta, pero no bloquea la entrega ni expone datos.
No cumple	No alcanza la meta y bloquea la entrega, o expone seguridad o integridad de datos.
Sin medir	Falta evidencia para calcular el indicador.

Índice de Calidad Global (ICG) = (indicadores en verde + 0,5 × indicadores en amarillo) ÷ indicadores medidos × 100. Meta de entrega propuesta: ICG ≥ 80 % y ningún indicador en rojo de seguridad.

2. Línea base de tamaño del producto

Conteo automático sobre los archivos entregados. Se usa como base para la densidad de defectos y la densidad de comentarios.

Archivo	Líneas	Código	Comentarios	% coment.
Frontend				
index.php	1 001	944	8	0,8 %
assets/js/app.js	1 376	1 301	22	1,7 %
features/feedback/feedback.js	265	249	0	0,0 %
assets/css/styles.css	2 376	1 961	13	0,7 %
features/feedback/feedback.css	87	74	0	0,0 %
Backend				
backend/endpoints/api.php	718	649	37	5,4 %
backend/endpoints/reporte.php	218	165	28	14,5 %
backend/libraries/xlsx.php	258	214	22	9,3 %
backend/services/mailer.php	87	73	6	7,6 %
backend/config/config.php	53	43	4	8,5 %
features/feedback/endpoint.php	126	111	0	0,0 %
Base de datos				
database/bdcatalogo.sql	638	471	56	10,6 %
features/feedback/schema.sql	16	16	0	0,0 %
Total	**7 219**	**6 271**	**196**	

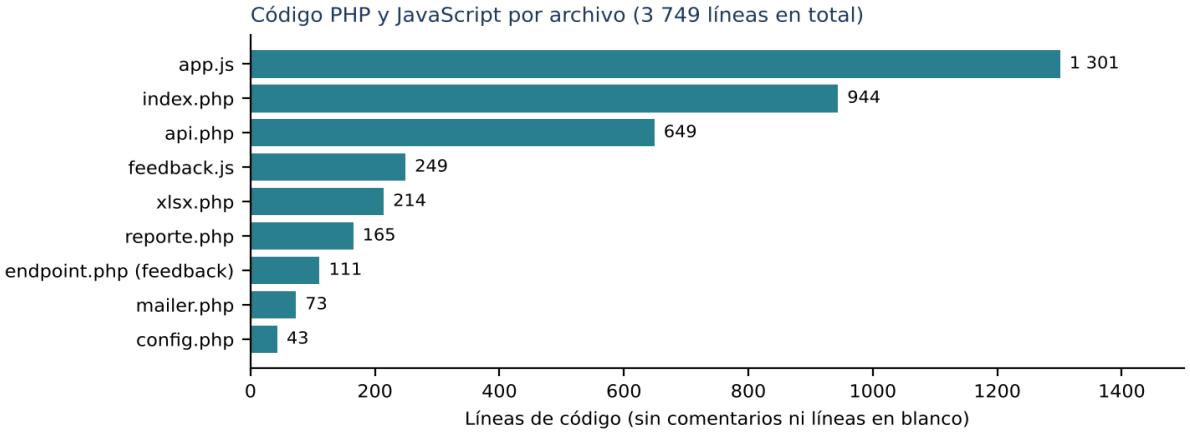


Figura 2. Distribución de las líneas de código PHP y JavaScript (base de 3,75 KLOC).

Otros datos de la línea base: 48 acciones en api.php (13 públicas, 15 de cliente y 20 de administrador); 22 procedimientos almacenados y 4 vistas; 6 tablas en database/bdcatalogo.sql, más 2 creadas en tiempo de ejecución por migrar() (transaccion_digital y comprobante) y 1 por el módulo feedback (feedback_reportes); 57 funciones JavaScript; 1 dependencia externa (PHPMailer v7.1.1).

3. Indicadores de calidad del producto

Fuentes: código entregado, bdcatalogo.sql, Verificación de RF (50 RF) y evaluación de RNF (16 RNF). Resultados en negrita.

ID	Indicador	Fórmula	Meta	Resultado	Estado
IP-01	Cumplimiento estricto de RF	RF «Cumple» ÷ RF revisados × 100	≥ 90 %	42 / 50 = **84 %**	**En riesgo**
IP-02	Cumplimiento ponderado de RF	(Cumple + 0,5 × Parcial) ÷ RF revisados × 100	≥ 95 %	(42 + 3,5) / 50 = **91 %**	**En riesgo**
IP-03	Cobertura del control de acceso	Acciones no públicas con needLogin() o needAdmin() ÷ acciones no públicas × 100	100 %	35 / 35 = **100 %** (15 de cliente y 20 de administrador)	**Cumple**
IP-04	Consultas parametrizadas	Consultas con datos del usuario que usan sentencias preparadas ÷ total × 100	100 %	**100 %**: todo pasa por q() (PDO prepare); 0 concatenaciones; las 7 llamadas exec() son DDL fijo de migrar()	**Cumple**
IP-05	Contraseñas provisionales en el código	Ocurrencias de la clave «123456» en api.php + hashes de ejemplo en el .sql	0	**4** en api.php (login y cambiar_password) y **5** hashes de ejemplo	**No cumple**
IP-06	Controles web defensivos	Controles presentes ÷ 3 (token CSRF, flags HttpOnly/SameSite de la cookie, límite de intentos de login)	3 / 3	**0 / 3**; el límite de frecuencia existe solo en el módulo de feedback	**No cumple**
IP-07	Puntos con riesgo de XSS	Atributos onclick que insertan texto escrito por usuarios	0	**3** (askDelete de producto, categoría y usuario)	**No cumple**
IP-08	Cumplimiento de RNF	(Cumple + Cumple con observación) ÷ 16 RNF × 100	≥ 80 %	8 / 16 = **50 %** (6 Cumple, 2 con observación, 6 Parcial, 2 Por verificar)	**No cumple**
IP-09	Fuente única del esquema de BD	Copias de bdcatalogo.sql idénticas	1 fuente	**2 copias distintas** (25 824 y 29 557 bytes); la de database/ no crea saldo_digital, transaccion_digital ni comprobante (los crea migrar())	**No cumple**
IP-10	Densidad de comentarios (PHP + JS)	Comentarios ÷ (código + comentarios) × 100	≥ 10 %	127 / 3 876 = **3,3 %** (index.php 0,8 %; app.js 1,7 %)	**En riesgo**
IP-11	Complejidad ciclomática estimada (API)	Bloques de acción con CC > 10 ÷ bloques medidos × 100	≤ 10 %	6 / 46 = **13 %**; promedio 4,5, máximo 21 (confirmar_pedido)	**En riesgo**

ID	Indicador	Fórmula	Meta	Resultado	Estado
IP-12	Módulos de gran tamaño	Archivos PHP/JS con más de 500 líneas ÷ archivos PHP/JS × 100	0	3 / 9 = **33 %**: app.js (1 376), index.php (1 001) y api.php (718, con 48 acciones en un solo switch)	**En riesgo**
IP-13	Duplicación de código	Líneas repetidas (≥ 40 caracteres) ÷ líneas evaluadas × 100	≤ 5 %	33 / 1 003 = **3,3 %** (heurística por líneas)	**Cumple**
IP-14	Pruebas automáticas	Casos automáticos (PHPUnit, Jest u otro) y su cobertura	≥ 1 por servicio crítico (SRS: 70 %)	**0 casos**; composer.json solo incluye PHPMailer	**No cumple**
IP-15	Diseño responsive	Quiebres CSS definidos y ancho mínimo verificado	Verificado a 360 px	3 quiebres (520, 640 y 860 px) más reducción de movimiento e impresión; **sin prueba** a 320/360 px	**En riesgo**
IP-16	Accesibilidad básica	Imágenes con alt ÷ imágenes × 100; contraste mínimo 4,5:1	100 % y contraste verificado	alt 4 / 4 = **100 %**; 12 aria-label, 4 aria-live, 2 role=dialog; contraste sin medir (RNF-14 Parcial)	**En riesgo**
IP-17	Validación de longitudes en servidor	RF con límite de longitud sin validar en servidor	0	**4 RF** (RF-07, 18, 31 y 32); la interfaz tiene 26 campos con maxlength y el exceso devuelve «Error de base de datos»	**En riesgo**
IP-18	Tiempo de respuesta (p95)	Percentil 95 con 50 peticiones concurrentes y 1 000 productos	Catálogo < 500 ms; pedido < 1 s	**Sin medir** (escenarios E1, E2 y E4 de JMeter pendientes, RNF-08)	**Sin medir**

Interpretación

- **Seguridad es el área más débil.** Los controles estructurales son buenos (IP-03 e IP-04), pero faltan los defensivos (IP-06) y sigue la puerta trasera de la clave 123456 (IP-05). Mientras IP-05 esté abierto, cualquier cuenta de ejemplo del .sql puede entrar con una clave conocida.
- **El riesgo de XSS (IP-07)** está en los botones de eliminar del panel: el nombre del producto, la categoría o el usuario se inserta dentro de un onclick. Como el nombre de un usuario lo escribe cualquier persona al registrarse, el código escrito ahí se ejecutaría en el navegador del administrador.
- **El esquema de datos no tiene una sola fuente (IP-09).** La app depende de migrar() para completar lo que falta en database/bdcatalogo.sql; funciona, pero oculta el diseño real y se ejecuta en cada sesión nueva.
- **Mantenibilidad intermedia.** La complejidad promedio es baja, pero confirmar_pedido (CC 21) concentra mucha lógica, api.php y app.js son archivos muy grandes y no hay pruebas automáticas (IP-14), por lo que cada cambio depende de repetir las pruebas manuales.
- **Funcionalidad sólida.** 91 % ponderado; los incumplimientos son puntuales (RF-10 devuelve 200 en vez de 401; RF-30 sin pantalla de recarga para el administrador).

4. Indicadores del proceso de calidad

Fuentes: documento Control de Calidad Técnico y Gestión de Versiones (17 casos de prueba, 8 Pull Requests, 7 errores) y revisión del repositorio.

ID	Indicador	Fórmula	Meta	Resultado	Estado
IR-01	Tasa de éxito de pruebas	Casos aprobados ÷ casos ejecutados × 100	100 %	17 / 17 = **100 %**	**Cumple**
IR-02	Cobertura de RF por casos de prueba	RF del SRS con al menos un caso ÷ 20 RF del SRS × 100	≥ 80 % (RF Alta)	8 / 20 = **40 %**; prioridad Alta 8 / 15 = 53 %. Sin caso: RF-04, 05, 06, 07, 12, 14, 15, 16, 17, 18, 19 y 20	**No cumple**
IR-03	Eficacia de detección de las pruebas (DDE)	Defectos hallados al ejecutar los CP ÷ defectos totales × 100	≥ 50 %	0 / 7 = **0 %**: los 7 INC salieron de revisar el repositorio, ninguno de los CP	**No cumple**
IR-04	Aprobación de Pull Requests	PR aprobados ÷ PR revisados × 100	≥ 80 %	5 / 8 = **62,5 %**; rechazados: PR-1 (conflictos), PR-4 (credenciales) y PR-7 (archivos excluidos)	**En riesgo**
IR-05	Independencia de la revisión	PR revisados por alguien distinto del autor ÷ PR × 100	100 %	8 / 8 = **100 %** (Danna 3, Camilo 3, Juan 2); los 8 PR tienen el mismo autor	**Cumple**
IR-06	Densidad de defectos	Defectos registrados ÷ KLOC de PHP + JS (3,75 KLOC)	≤ 1 por KLOC	7 / 3,75 = **1,87**; con los 4 hallazgos nuevos (INC-08 a 11) sube a 2,93	**En riesgo**

ID	Indicador	Fórmula	Meta	Resultado	Estado
IR-07	Defectos Críticos o Altos abiertos	INC de severidad Crítica o Alta sin verificar	0	**4** (INC-01, 03, 04 y 05); en el código entregado se confirma abierto INC-05	**No cumple**
IR-08	Cierre comprobado de defectos	INC resueltos y comprobados en el código ÷ INC registrados × 100	100 %	2 / 7 = **29 %** (INC-01 e INC-02); 2 parciales, 1 abierto y 2 no verificables sin el historial de Git	**No cumple**
IR-09	Higiene del repositorio	Archivos fuera de lugar o reglas de .gitignore incorrectas	0	**2 archivos .patch** (38 456 bytes) en la raíz y .gitignore con *.sql sin regla para database/*.sql	**En riesgo**
IR-10	Coherencia de la configuración por defecto	Discrepancias entre config.php y README	0	**1**: DB_HOST por defecto 127.0.0.1:8080 anula el puerto 3306 que indica el README	**No cumple**
IR-11	Trazabilidad y vigencia documental	Documentos con numeración de requisitos igual al SRS; secciones del SRS desactualizadas	3 de 3; 0 secciones	**1 de 3** (CP usa los 20 RF del SRS; la Verificación usa 50 RF; el Excel usa 16 RNF frente a 8 del SRS); 3 secciones del SRS marcadas como desactualizadas (3.2, 4 y 5)	**En riesgo**

Interpretación

- **Pasar el 100 % de las pruebas no significa estar libre de errores.** CP-05 y CP-06 (login) aprueban aunque la clave 123456 siga aceptada; ningún CP intenta una cuenta de ejemplo, un nombre con comillas ni un puerto de base de datos distinto. La eficacia de detección es 0 % (IR-03).
- **Hay 12 RF sin ningún caso de prueba**, 7 de ellos de prioridad Alta (RF-05, 06, 14, 15, 16, 17 y 18): catálogo y búsqueda, precios e inventario, categorías, pedidos de administrador, usuarios y branding.
- **La revisión de código sí funciona como filtro:** los 3 PR rechazados detectaron conflictos, credenciales y archivos que no debían subirse. Falta repartir la autoría: los 8 PR son del mismo autor.

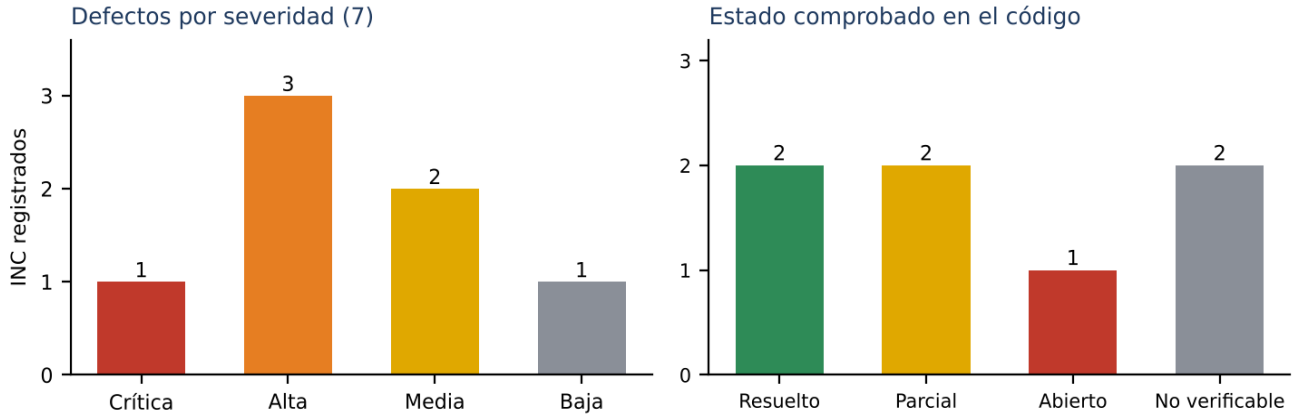


Figura 3. Defectos registrados por severidad y estado comprobado contra el código entregado.

5. Verificación de los defectos INC contra el código entregado

El documento de Control de Calidad deja los 7 errores en estado «Reportado». Se contrastó cada uno con el código del zip:

ID	Severidad	Evidencia en el código entregado	Estado comprobado
INC-01	Crítica	config.php lee DB_* y MAIL_* de variables de entorno; no hay claves en el código.	**Resuelto** en código. Falta rotar la contraseña de aplicación de Google: sigue en el historial de Git.
INC-02	Media	Existe .gitignore con vendor/ y el zip no contiene vendor/.	**Resuelto**. Conviene añadir *.swp y *.patch.
INC-03	Alta	El zip no incluye .git ni archivos .swp.	No verificable con el entregable.
INC-04	Alta	Anexos/ reemplaza a «Carpeta docu»; sin .git no se ve el estado del commit.	No verificable con el entregable.
INC-05	Alta	api.php conserva la clave 123456 en login y cambiar_password; el .sql tiene 5 hashes «hashEjemplo».	**Abierto**.
INC-06	Media	El error de conexión ya es genérico y el detalle va a error_log() en config.php; pero migrar() aún devuelve el mensaje de MySQL al usuario.	**Parcial**.

ID	Severidad	Evidencia en el código entregado	Estado comprobado
INC-07	Baja	README con requisitos, instalación y variables; faltan usuarios de prueba y quedan 2 .patch en la raíz.	**Parcial**.

Hallazgos nuevos propuestos

Se recomienda agregarlos a la tabla de la sección 6 del Control de Calidad con los números siguientes, indicando el caso de prueba que los habría detectado:

ID	Severidad	Descripción	Corrección sugerida
INC-08	Alta	DB_HOST por defecto es «127.0.0.1:8080»; el código separa el puerto del texto y anula DB_PORT (3306). Sin variables de entorno, una instalación limpia en XAMPP no conecta, pese a lo que dice el README.	Cambiar el valor por defecto a 127.0.0.1:3306.
INC-09	Media	Hay dos copias distintas de bdcatalogo.sql. La de database/ (la que indica el README) no crea saldo_digital, transaccion_digital ni comprobante ni las columnas de envío; esas piezas las crea migrar().	Dejar una sola copia completa y mantener migrar() solo para actualizar bases antiguas.
INC-10	Alta	Riesgo de XSS: askDelete() recibe en el onclick el nombre de producto, categoría o usuario; el navegador revierte ' a comilla antes de ejecutar el JavaScript.	Pasar el id por atributo data-* y asignar el evento con addEventListener.
INC-11	Baja	El .gitignore ignora *.sql y la regla !database/ no reincluye database/bdcatalogo.sql.	Usar !database/*.sql y comprobar con git check-ignore -v.

6. Acciones de mejora priorizadas

Prioridad	Acción	Indicadores que mejora	Criterio de cierre
1 · Antes de entregar	Eliminar la rama de la clave 123456 en login y cambiar_password y reemplazar los hashes de ejemplo; repetir CP-05 y CP-06.	IP-05, IR-07, IR-08	0 ocurrencias de 123456 en api.php
1 · Antes de entregar	Corregir DB_HOST por defecto (INC-08) y unificar bdcatalogo.sql (INC-09).	IR-10, IP-09	Instalación limpia funciona siguiendo el README
1 · Antes de entregar	Corregir el XSS de askDelete (INC-10) y rotar la contraseña de aplicación de Google (INC-01).	IP-07, IR-08	0 puntos de XSS; clave rotada
2 · Siguiente versión	Agregar token CSRF, session_set_cookie_params (HttpOnly, SameSite) y límite de intentos de login.	IP-06	3 / 3 controles
2 · Siguiente versión	Escribir casos de prueba para los 12 RF sin cobertura (primero los 7 de prioridad Alta) y casos negativos de seguridad.	IR-02, IR-03	Cobertura RF Alta ≥ 80 %
2 · Siguiente versión	Corregir RF-10, RF-30, RF-45, RF-29 y las validaciones de longitud en servidor.	IP-01, IP-02, IP-17	Cumplimiento estricto ≥ 90 %
3 · Mejora continua	Separar confirmar_pedido y el switch de api.php por módulos; comentar funciones críticas; PHPUnit mínimo para moverSaldo, totalPedido y confirmar_pedido.	IP-10, 11, 12, 14	Comentarios ≥ 10 %; ≥ 3 pruebas automáticas
3 · Mejora continua	Medir rendimiento (JMeter E1, E2, E4), accesibilidad (contraste con Lighthouse o axe) y pantalla de 360 px.	IP-15, 16, 18	Sin indicadores «Sin medir»
3 · Mejora continua	Actualizar SRS (secciones 3.2, 4 y 5), alinear la numeración de RF/RNF entre documentos y mover los .patch a docs/.	IR-11, IR-09	3 de 3 documentos alineados

7. Plan de medición

Para que los indicadores sean una práctica y no un informe único, se propone medirlos así:

Familia	Indicadores	Frecuencia	Fuente o herramienta sugerida
Tamaño, complejidad y duplicación	IP-10 a IP-13	En cada entrega a main	phploc o PHPMetrics (PHP); ESLint (JS); conteo por script
Seguridad y análisis estático	IP-03 a IP-07	En cada Pull Request	PHPStan o Psalm; composer audit; lista de revisión (sección 4 del Control de Calidad)
Pruebas	IP-14, IR-01 a IR-03	En cada PR y cada entrega	Matriz CP ↔ RF; PHPUnit
Requisitos	IP-01, IP-02, IP-08, IP-17	En cada entrega	Verificación de RF y Excel de RNF actualizados

Familia	Indicadores	Frecuencia	Fuente o herramienta sugerida
Rendimiento y accesibilidad	IP-15, IP-16, IP-18	Antes de cada entrega	JMeter; Lighthouse o axe; DevTools a 360 px
Proceso y defectos	IR-04 a IR-11	Semanal y por entrega	Registro de INC y Pull Requests de GitHub

Metas para la versión 1.3: ICG ≥ 70 %; 0 defectos Críticos o Altos abiertos; cobertura de RF de prioridad Alta ≥ 80 %; eficacia de detección (DDE) ≥ 50 %; al menos 3 pruebas automáticas; 0 indicadores «Sin medir».

8. Limitaciones y consistencia de los datos

- **Medición estática.** No se ejecutó la tienda ni se usó una base MySQL; el rendimiento y la compatibilidad no se midieron. Las líneas y la complejidad se calcularon con un script propio (conteo de puntos de decisión), así que PHPMetrics o PHPStan pueden dar cifras algo distintas. Sirven como línea base para comparar entre versiones.
- **El zip no incluye .git ni vendor/.** Por eso INC-03 e INC-04 no se pueden comprobar y PHPMailer no se auditó.
- **Inconsistencias en los documentos de origen** que conviene corregir: (a) en el Control de Calidad la columna Estado dice «Funcional» pero la leyenda usa Aprobado, Fallido y Pendiente; (b) el resumen indica 7 errores verificados mientras la tabla los muestra «Reportado» y 4 Críticos o Altos abiertos: aquí se usó la tabla y la comprobación en el código; (c) los CP usan los 20 RF del SRS y la Verificación usa 50 RF; (d) el PR-6 figura aprobado («se actualizó bdcatalogo.sql») aunque la copia de database/ no incluye las tablas de dinero digital; (e) las secciones 3.2, 4 y 5 del SRS aún describen NestJS, React y PostgreSQL, como advierte el propio SRS.
- **Metas propuestas.** Salvo las que vienen del SRS, las metas son propuestas del análisis y deben validarse con el equipo.

9. Conclusión y decisión sugerida

TechnoStore tiene una base técnica sólida en lo estructural (acceso por rol, SQL parametrizado, transacciones con bloqueo y trazabilidad del dinero digital) y cumple el 91 % de sus requisitos funcionales. Aun así, con un ICG de **39,3 %** y 11 indicadores en rojo, no cumple todavía el criterio de entrega definido por el equipo (casos aprobados y ningún error Crítico o Alto sin verificar). Los cinco puntos de la prioridad 1 son pequeños de corregir y elevarían el ICG de forma notable.

Concepto	Valor
Indicadores medidos / total	28 / 29
En verde / amarillo / rojo / sin medir	5 / 12 / 11 / 1
Índice de Calidad Global (ICG)	**39,3 %** (meta de entrega: 80 %)
Defectos Críticos o Altos abiertos	4 registrados (1 confirmado en el código) + 2 hallazgos nuevos de severidad Alta (INC-08 e INC-10)
Decisión sugerida	☐ Aprobado para entrega ☒ **No aprobado** (pendiente: acciones de prioridad 1 y nueva medición)